

팀장 | 송정훈

팀원 | 민동찬

팀원 | 최준영

팀원 | 배호민

2026.05.29

목차

01 게임 소개
02 코어시스템/유틸
03 무기
04 몬스터
05 아이템
06 시연/Q&A

게임 소개

다양한 컨셉의 귀여운 캐릭터들이 **대량의 몬스터를 처치하며 성장**하고,
장애물을 돌파해 보스를 공략하는 캐주얼 서바이벌 액션 게임

핵심 재미 레퍼런스



- 대량의 몬스터를 쓸어 내는 시원한 전투감
- 3D 공간에서 구현된 뱀서라이크식 성장 액션

메가봉크



- 다양한 장애물 돌파
- 실패해도 웃긴 유쾌한 파티게임 감성

폴가이즈

차별점

단순히 오래 살아남는 뱀서라이크가 아닌,
성장 → 장애물 돌파 → 보스 공략 으로 이어지는 **목표형 서바이벌 게임**

게임 플로우

메인 진행

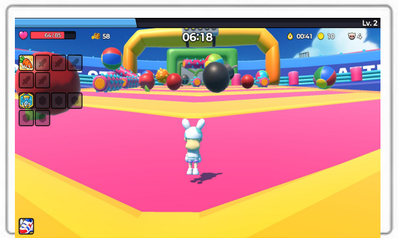
1 타이틀



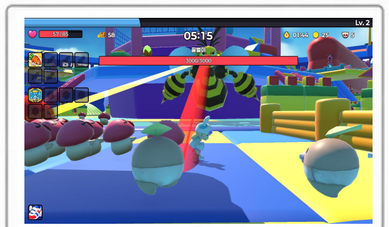
2 캐릭터 선택



3 장애물 극복



4 스테이지 보스 배틀



6종류의 장애물 모두 극복 시
파이널 보스 등장

반복 성장 루프

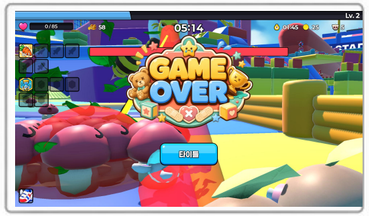
5 레벨업 시 강화 선택



6 보물상자 보상

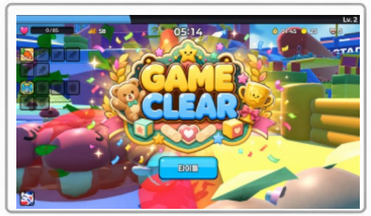


7 게임오버



플레이어 HP 모두 소진 시

8 게임클리어



파이널 보스 격파 시

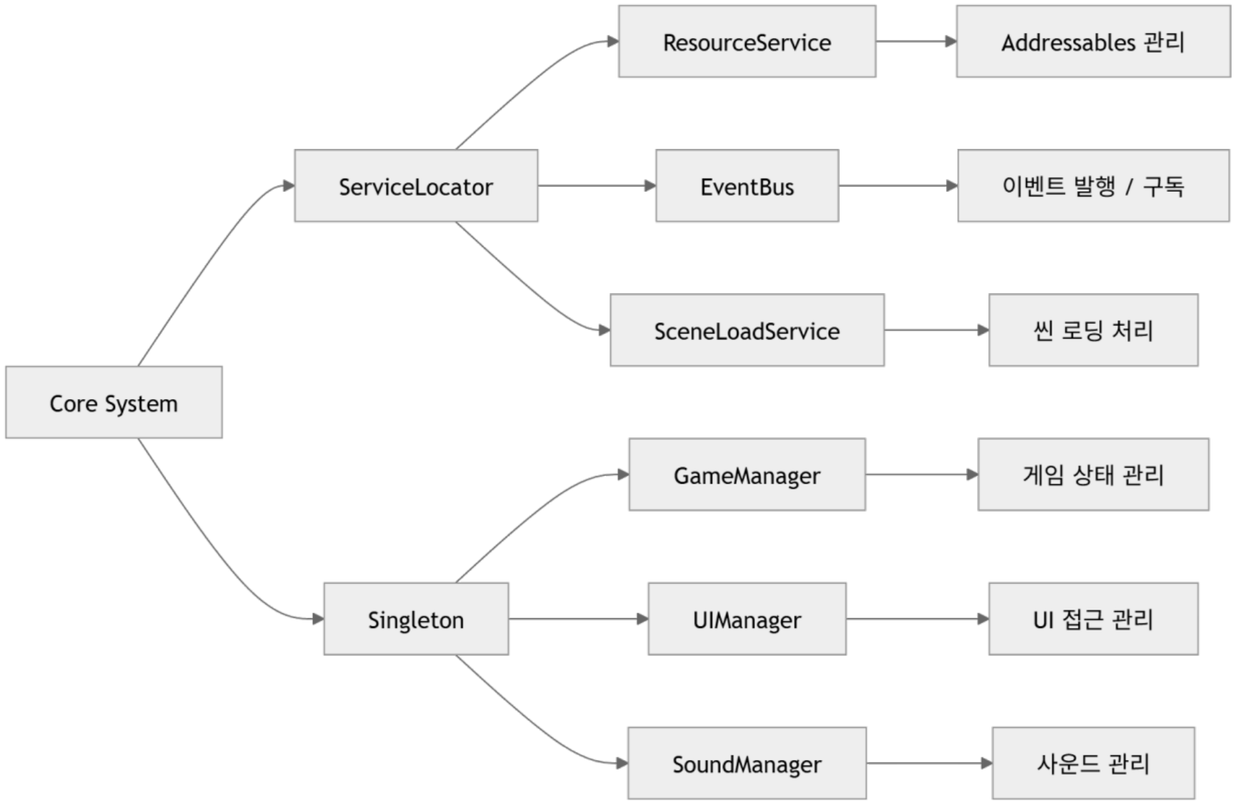
코어시스템/유틸

송정훈

코어 시스템 아키텍쳐

코어 시스템은 ServiceLocator와 Singleton을 병행하여 설계

- 서비스 성격의 기능은 ServiceLocator로 관리하고, Unity 매니저 객체는 Singleton으로 구현



특징

- 인터페이스 기반으로 사용
- 초기화 시점에 코어 서비스를 등록하여 관리
- 테스트 용이 및 유연한 구조 제공

특징

- 어디서든 Instance로 간편하게 접근
- 중복 인스턴스 생성을 방지
- 씬 유지 여부에 따라 구조 분리

Singleton 로직 공통화

```
public class GameManager : MonoBehaviour
{
    참조 7개
    public static GameManager Instance { get; private set; }

    Unity 메시지 | 참조 0개
    private void Awake()
    {
        if (Instance == null)
        {
            Instance = this;
            DontDestroyOnLoad(gameObject);
            return;
        }

        Destroy(gameObject);
    }
}
```

```
public class SoundManager : MonoBehaviour
{
    참조 4개
    public static SoundManager Instance { get; private set; }

    Unity 메시지 | 참조 0개
    private void Awake()
    {
        if (Instance == null)
        {
            Instance = this;
            DontDestroyOnLoad(gameObject);
            return;
        }

        Destroy(gameObject);
    }
}
```

Instace / Awake / 중복 제거 로직 등이 매번 반복됨

```
public abstract class MonoSingleton<T> : MonoBehaviour
    where T : MonoBehaviour
{
    private static T _instance;

    참조 34개
    public static T Instance
    {
        get
        {
            if (_instance == null)
                _instance = FindAnyObjectByType<T>();

            return _instance;
        }
    }

    Unity 메시지 | 참조 2개
    protected virtual void Awake()
    {
        T current = this as T;

        if (_instance == null)
            _instance = current;

        if (_instance == current)
        {
            DontDestroyOnLoad(gameObject);
            return;
        }

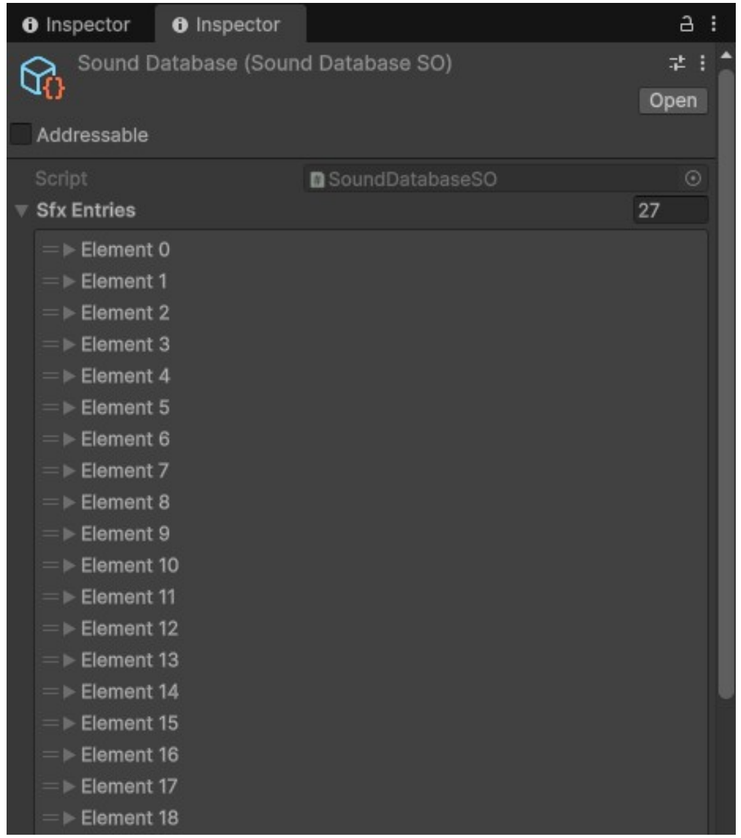
        Destroy(gameObject);
    }
}
```

```
public class GameManager : MonoSingleton<GameManager>
public class SoundManager : MonoSingleton<SoundManager>
```

제네릭 기반 MonoSingleton<T>로 공통화

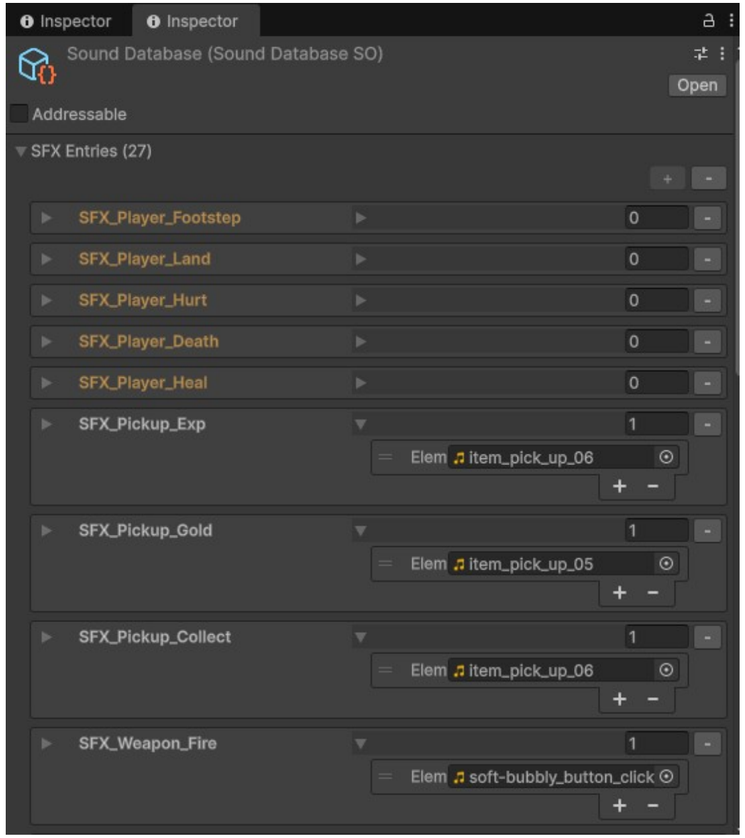
추가로 DontDestroyOnLoad를 사용하지 않는 싱글톤 로직도 구현 가능

커스텀 인스펙터



문제

List 데이터가 Element 단위로만 표시되어
내부 정보를 확인하려면 항목을 직접 펼쳐봐야 함



해결

핵심 데이터를 Inspector에 직접 노출하여
데이터 상태를 한눈에 확인할 수 있도록 개선

Extension Method (확장 메서드)

```
public class PlayerController : MonoBehaviour
{
    private Rigidbody _rigidbody;


    private void Awake()
    {
        _rigidbody = GetComponent<Rigidbody>();

        if (_rigidbody == null)
        {
            _rigidbody = gameObject.AddComponent<Rigidbody>();
        }
    }
}
```

```
public class HitBox : MonoBehaviour
{
    private BoxCollider _collider;

    Unity 메시지 | 참조 0개
    private void Awake()
    {
        _collider = GetComponent<BoxCollider>();

        if (_collider == null)
        {
            _collider = gameObject.AddComponent<BoxCollider>();
        }
    }
}
```

GetComponent 후 null 체크,
AddComponent 처리 코드가 반복됨

```
public static class ComponentExtensions
{
    참조 7개
    public static T GetOrAddComponent<T>(this GameObject gameObject)
        where T : Component
    {
        if (gameObject == null)
            return null;

        T component = gameObject.GetComponent<T>();
        if (component != null)
            return component;

        return gameObject.AddComponent<T>();
    }
```

GetOrAddComponent<T>()로
컴포넌트 조회와 생성을 하나의 메서드로 공통화

```
public class PlayerController : MonoBehaviour
{
    private Rigidbody _rigidbody;

    Unity 메시지 | 참조 0개
    private void Awake()
    {
        _rigidbody = gameObject.GetOrAddComponent<Rigidbody>();
    }
}
```

코드 간소화

무기

배호민

무기 - 패턴 분류

무기 시스템은 **공격 로직에 따라 타입을 구분**해 다양한 전투 패턴을 **확장 가능하게 구성**

ProjectileWeapon	HomingWeapon	OrbitWeapon	AuraWeapon	ChainLightningWeapon
				
적을 향해 투사체를 발사하는 무기	적을 추적하는 유도 투사체 무기	주변을 회전하며 피해를 주는 무기	주변 적에게 지속 피해를 주는 오라형 무기	적 사이를 연쇄적으로 타격하는 번개형 무기

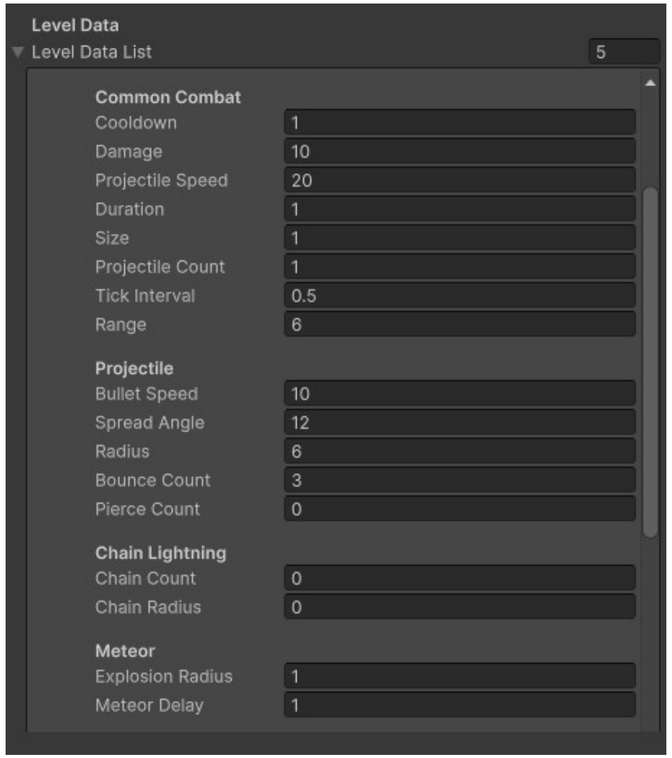
MeteorWeapon	DroneWeapon	BoomerangWeapon	BouncyProjectileWeapon	PiercingRaycastWeapon
				
범위에 오브젝트를 떨어뜨리는 광역 무기	드론이 자동으로 공격하는 소환형 무기	되돌아오며 적을 타격하는 무기	튕기며 여러 적을 공격하는 투사체 무기	직선상 적을 관통 타격하는 무기

무기 - 레벨별 강화 시스템

각 무기는 WeaponLevelData로 레벨별 성능을 정의하고, 이를 **WeaponDefinitionSO에서 관리해 수치 조정과 확장이 쉽도록 구성**

강화 적용 흐름





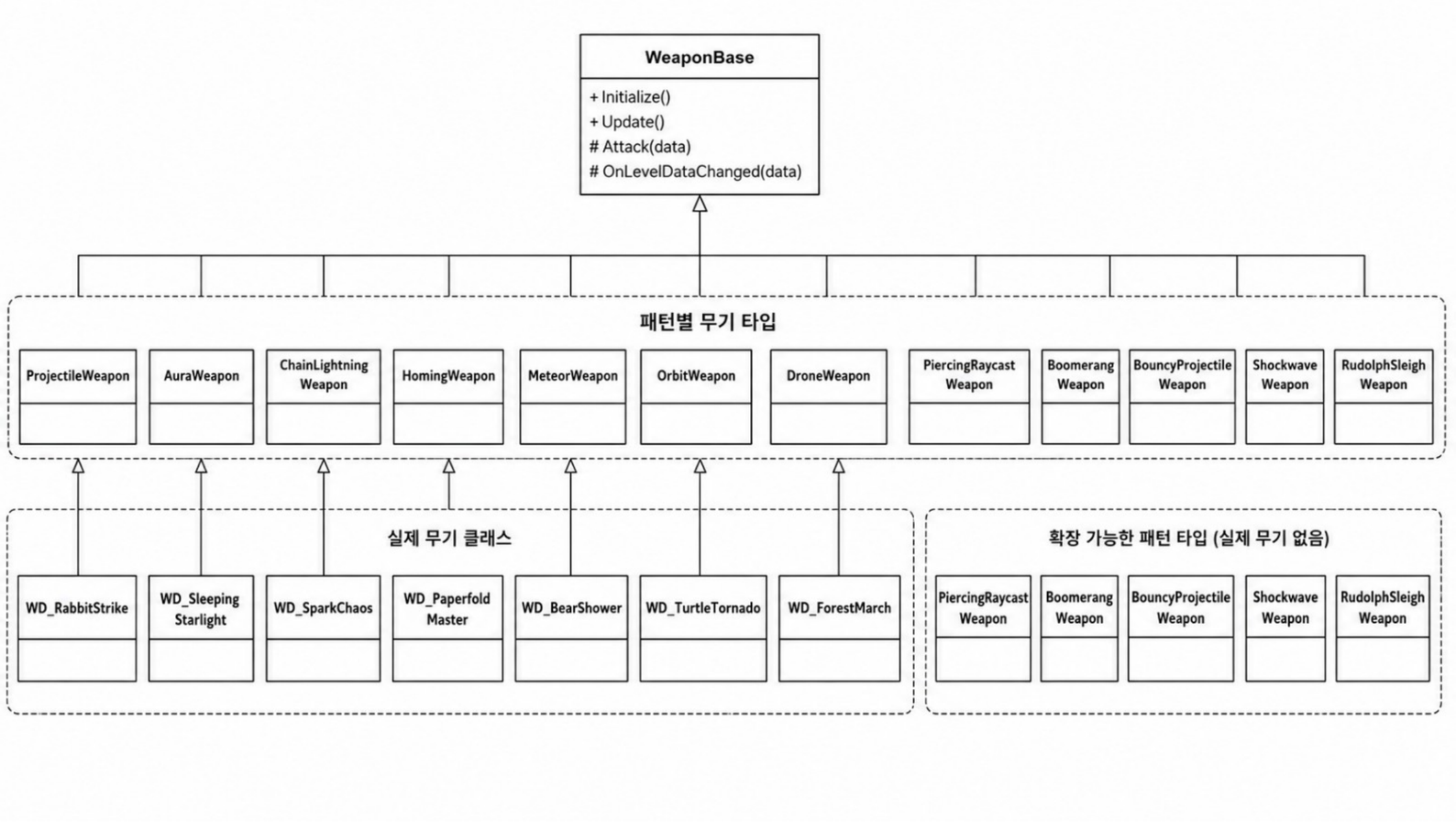
레벨별 스텟 정의

각 레벨의 변동 스탯을
개별적으로 설정

코드 수정 없이 SO 데이터만으로 밸런스 조절 가능

무기 - 클래스 구조

공통 기능은 부모 클래스에서 처리하고, 무기별 공격 방식만 자식 클래스에서 구현하도록 분리

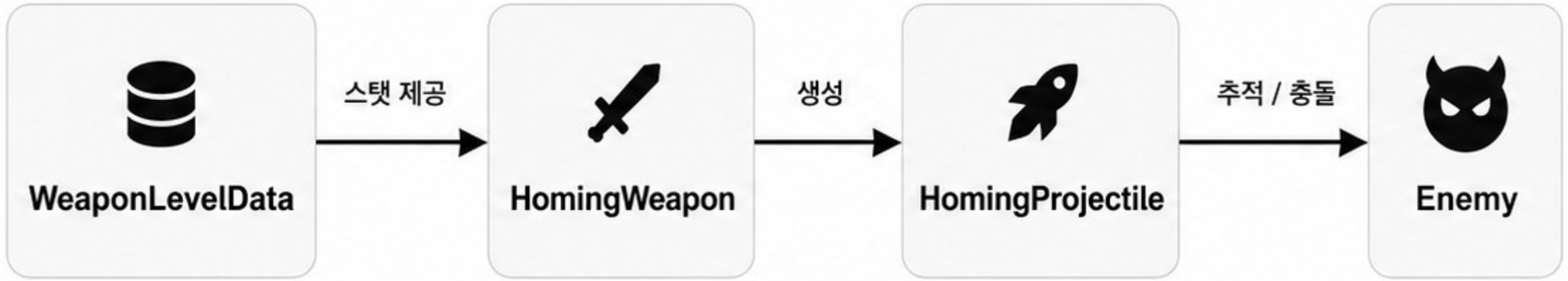


핵심 효과

- 공통 로직 재사용
- 패턴별 책임 분리
- 신규 무기 확장에 용이

무기 - 종이비행기 (HomingWeapon)

구조



동작 흐름

1. Attack 호출
2. 가장 가까운 적 탐색
3. 호밍 투사체 생성
4. 타켓 방향 추적
5. 충돌 시 데미지 적용

설계 포인트

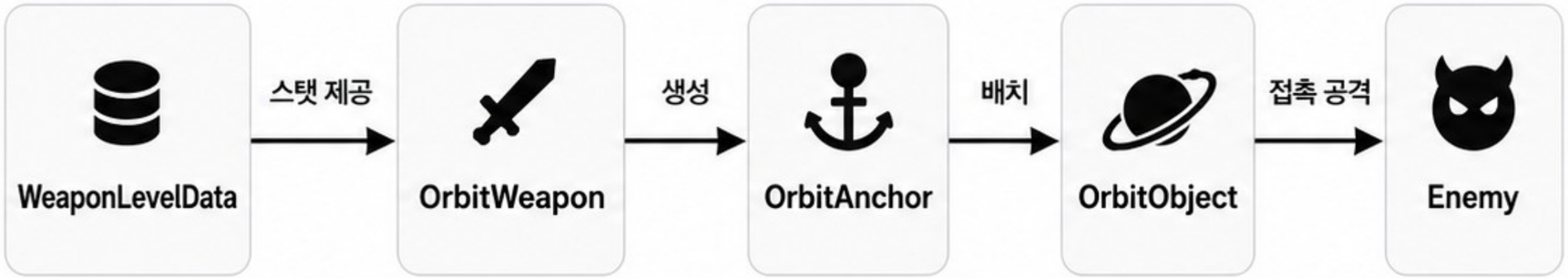
타켓 탐색
가장 가까운 적 우선 추적

부드러운 추적
회전 보간으로 자연스러운 이동

데이터 기반 제어
속도·데미지를 LevelData로 관리

무기 - 쉘 스핀 (OribitWeapon)

구조



동작 흐름

1. Attack 호출
2. OrbitAnchor 생성
3. OrbitObject 배치
4. 플레이어 중심 공전
5. 접촉 시 데미지 적용

설계 포인트

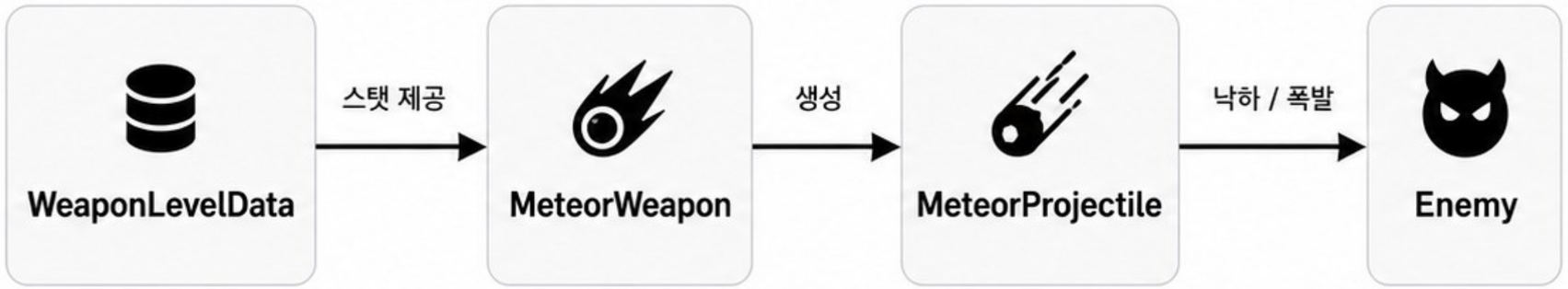
공전 반경 제어
Range로 궤도 거리 설정

재타격 간격 관리
Cooldown으로 동일 대상 재타격 제어

데이터 기반 제어
개수·속도·데미지를 LevelData로 관리

무기 - 젤리곰 드롭 (MeteorWeapon)

구조



동작 흐름

1. Attack 호출
2. 랜덤 낙하지점 선정
3. 경고 이펙트 생성
4. 메테오 낙하
5. 폭발 시 범위 데미지 적용

설계 포인트

랜덤 낙하지점
안전 거리 밖 임의의 위치 선정

범위 폭발 피해
ExplosionRadius 내 적에게 데미지

데이터 기반 제어
속도·폭발 범위·지연 시간을 LevelData로 관리

몬스터

최준영

몬스터 (일반) - 타입별 소개



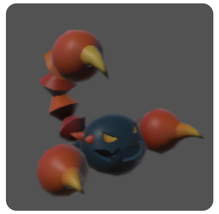

타입	한 줄 역할
일반	플레이어 기본 추격

BasicEnemy




타입	한 줄 역할
돌진	플레이어를 향해 돌진

DashEnemy



타입	한 줄 역할
원거리	사거리 내에서 원거리 공격

RangedEnemy




타입	한 줄 역할
레이져	피격 범위를 보여준 후 레이저 공격

LaserEnemy




타입	한 줄 역할
비행	빠른 속도로 비행

FlyingEnemy



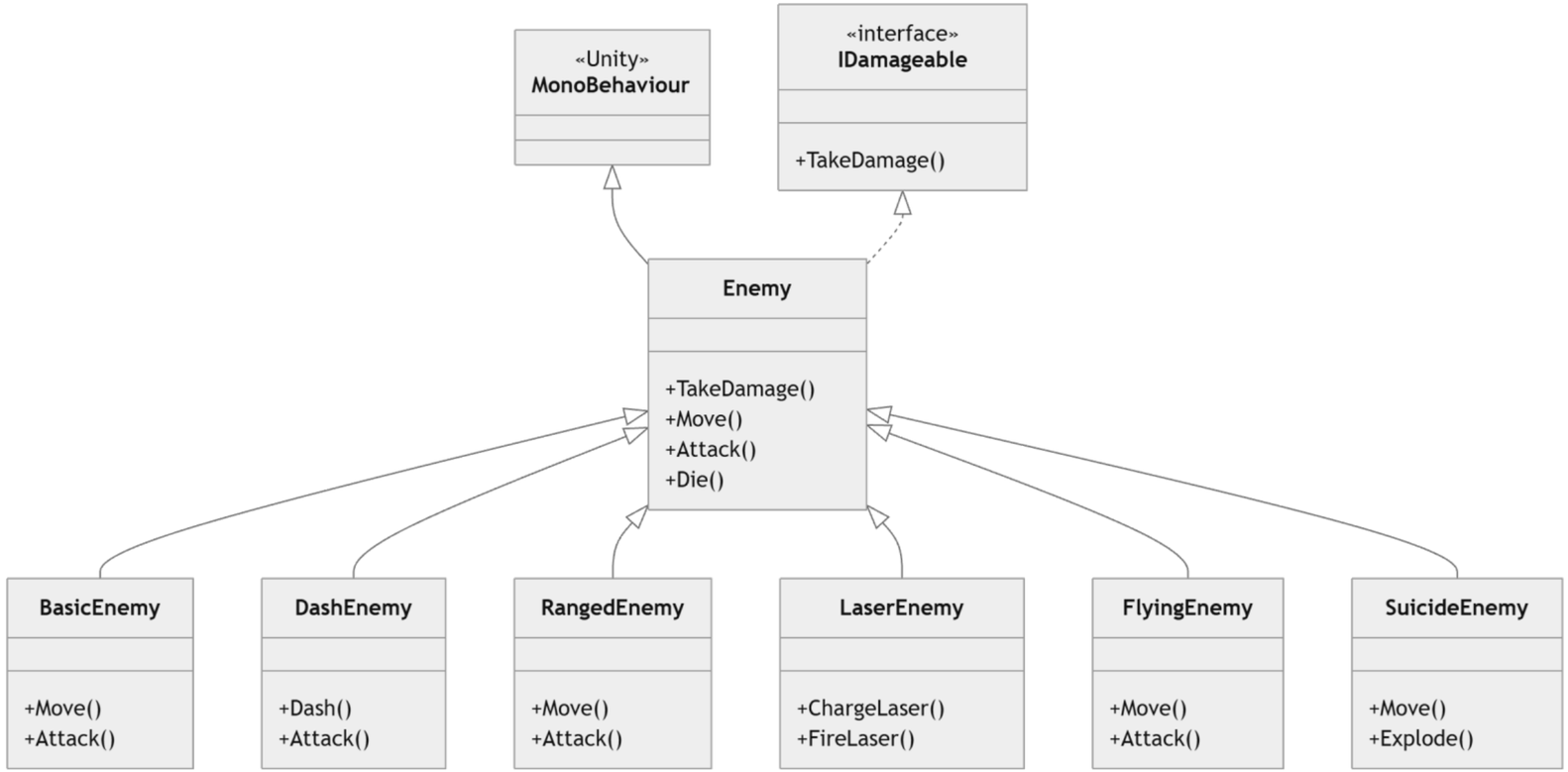
타입	한 줄 역할
자폭	플레이어와 근접 시 자폭 공격

SuicideEnemy

몬스터 (일반) - 상속 구조

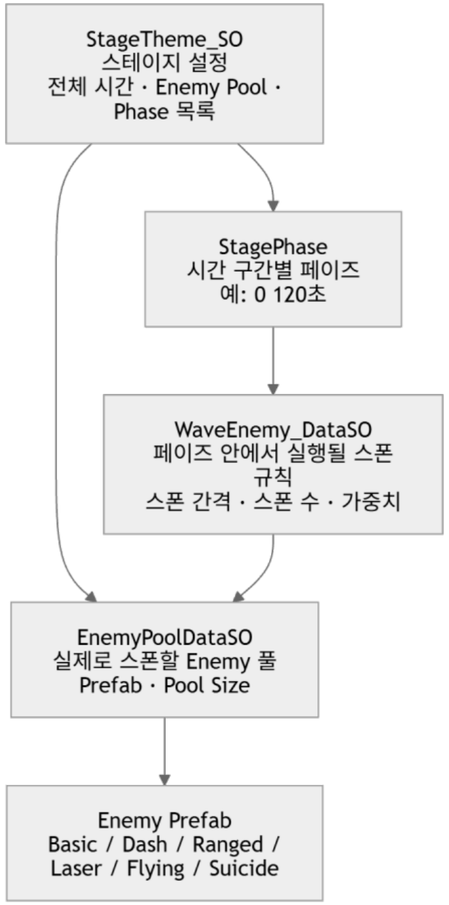
Enemy **공통 로직을 베이스**에 두고, 타입별 공격·이동 패턴만 **서브클래스로 확장**

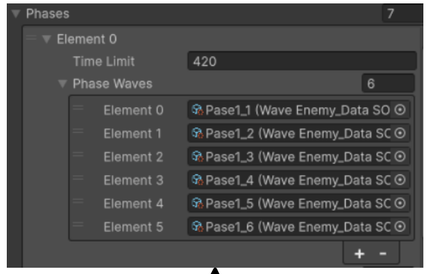
- 새 몬스터 추가 시 공통 코드 재작성 없이 패턴·프리팹만으로 확장 가능



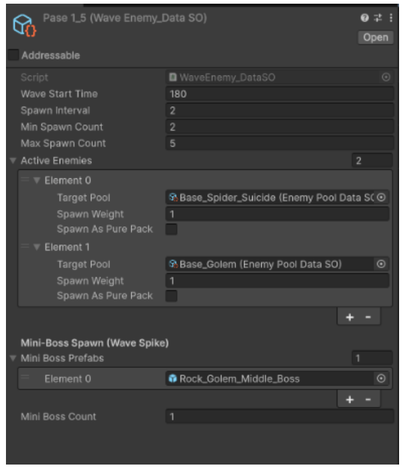
몬스터 스폰 - Stage / Wave 데이터 (SO)

스폰 조건을 SO(에셋)로 분리 → **디버깅·밸런스 관리 용이**





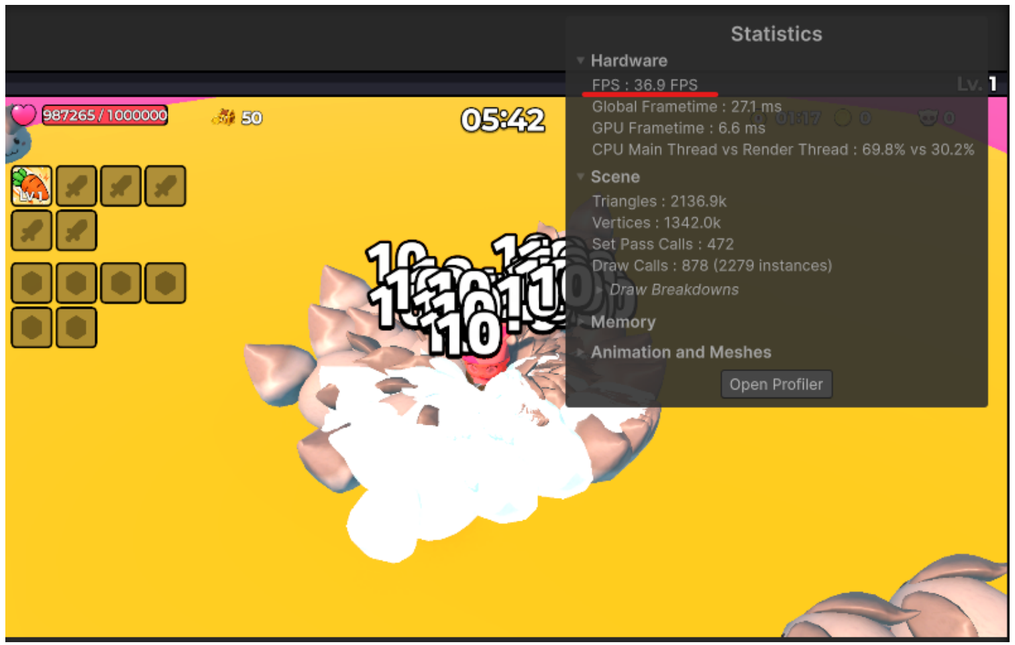
Wave SO 등록



설정 항목	한 줄 설명
웨이브 시작	페이즈 시작 후 N초에 이 웨이브 활성 (Wave Start Time)
스폰 규칙	N초마다 최소~최대 M마리 스폰 (Spawn Interval, Min/Max Count)
등장 몬스터	몬스터 풀(EnemyPoolDataSO) 등록 및 가중치(Weight)로 등장 비율 조절
미니보스	웨이브 시작 시 등장할 미니보스 종류·마리 수 설정 (Mini Boss Prefabs, Count)

몬스터 스폰 - SO 실행과 성능 최적화

- 앞서 정의한 **WaveSO** 조건을 **EnemyManager**가 읽어 몬스터를 필드에 스폰
- 동시에 활성화되는 Enemy가 많아지면 **프레임이 저하**되므로, **런타임(CPU) · 렌더링(GPU) 양쪽에서 최적화를 진행**



대량 Enemy 스폰 시 프레임 드랍

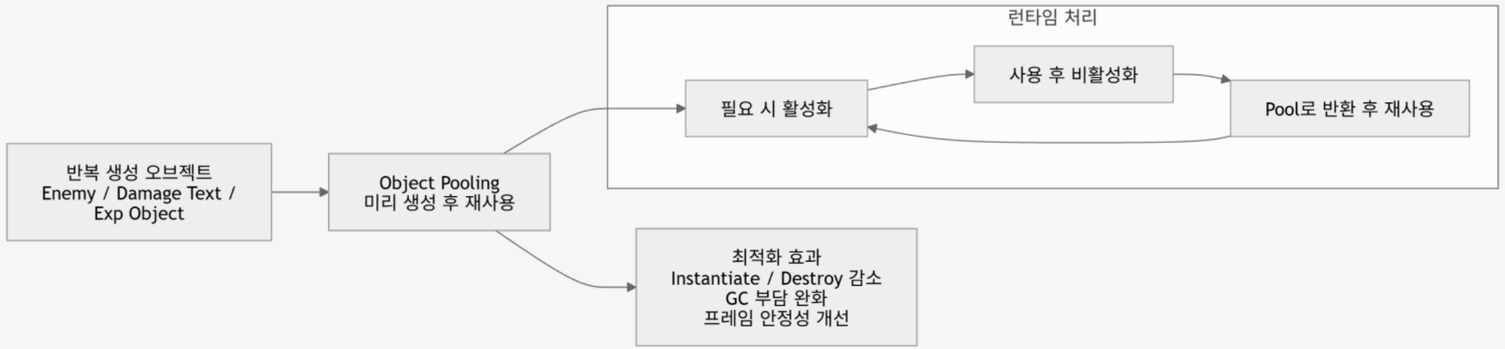
- **오브젝트 풀링** - Instantiate/Destroy 반복 대신 풀 재사용으로 GC 호출 감소
- **ManagedUpdate** - 개별 Update 제거, 매니저가 몬스터 AI 일괄 갱신
- **Grid 기반 공간 분할** - 물리·콜라이더 없이 몬스터 끼리 겹치지 않게 처리
- **Multi-Raycast 이동** - NavMesh 없이 3D 지형을 따라 플레이어 추격
- **VAT + GPU Instancing** - 동일한 메쉬를 가진 몬스터 다수 출현 시 드로우·애니메이션 비용 절감

몬스터 최적화 - 오브젝트 풀링

※ 문제

Instantiate / Destroy 반복 시 **GC가 자주 발생해 성능 저하 가능**

※ 해결



전투 중 반복적으로 생성되는 Enemy, DamageText, ExpObject 등에 대해 **타입별로 별도의 Pool**을 구성하여 Instantiate / Destroy 호출을 줄였다.

※ 결과

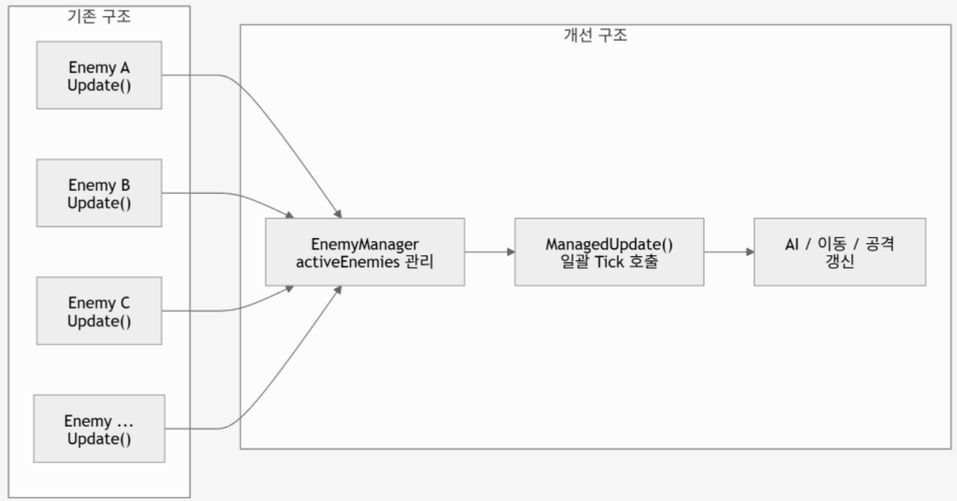
반복 생성 비용을 줄여 **GC 부담을 완화**하고, **대량 스폰 상황의 프레임 안정성 개선**

몬스터 최적화 - ManagedUpdate

※ 문제

수백 개의 Enemy가 각자 Update()를 실행하면 **Unity 메시지 호출 오버헤드와 CPU 연산 부담이 증가**

※ 해결



중앙 관리자인 EnemyManager가 활성화된 Enemy를 리스트로 관리하고, 매 프레임 **각 Enemy의 ManagedUpdate()를 일괄 호출**하도록 변경

※ 결과

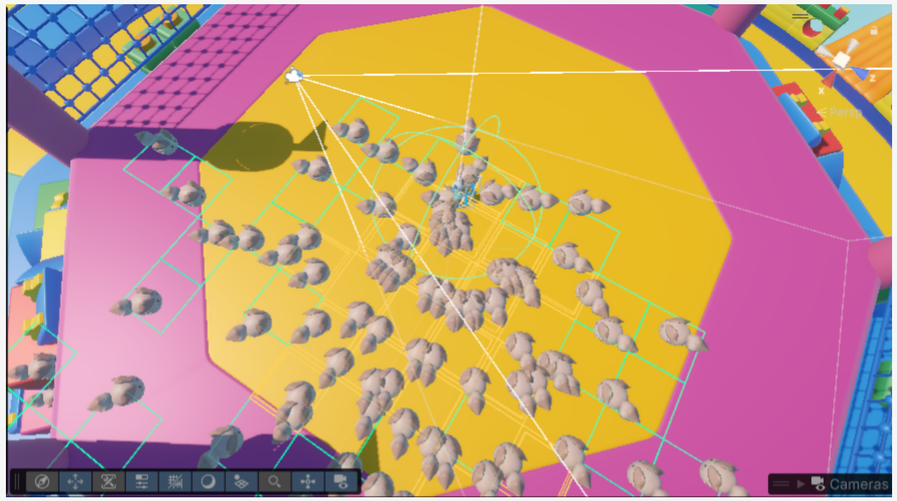
개별 Update 비용을 줄여 **CPU 연산 효율을 개선**하고 대량 스폰 상황에서도 **일관된 갱신 흐름을 유지**

몬스터 최적화 - Grid 기반 공간 분할(Spatial Partitioning)

※ 문제

대량 Enemy에 Unity 물리 기반 충돌 처리를 적용할 경우 **충돌 계산과 물리 콜백 비용이 누적되어 프레임 저하 발생**

※ 해결



1. 몬스터 위치를 **Grid Cell 단위**로 나누어 같은 Cell에 있는 몬스터들을 그룹화

2. 현재 Cell과 인접 Cell의 몬스터만 검사하여 분리(Separation) **계산 범위 축소**

3. 매 프레임마다 계산하지 않고 일정 주기(Interval)마다 처리되도록 구현

※ 결과

겹침 방지에 필요한 물리만 코드로 계산해 **Unity 물리 의존을 줄였고**, 가까운 Enemy만 처리해 **연산 최적화**를 달성

몬스터 최적화 - NavMesh 없는 3D 추적 로직

※ 문제

대량 Enemy에 **NavMeshAgent 기반 추적**을 적용하면 **경로 재계산과 회피 연산 비용이 누적되어 프레임 저하**가 발생

※ 해결



1. 플레이어 좌표 기반 XZ 추적 (수평)

- 플레이어 위치를 기준으로 **Y를 제외한 수평 방향(XZ) 을 계산해 이동**

2. 플레이어와의 Y 단차에 따른 조건부 Raycast

- Y 단차에 따라 Low Ray(단일·단거리) / High Multi-Raycast(장거리·다중)로 전환해, **상황에 맞는 Raycast만 사용해 비용 절감**

3. 다층 지형 Y 보정

- High Multi-Raycast에서 여러 Ground hit 중 **플레이어 Y와 가장 가까운 지면**을 선택해 **Y축 이동**

※ 결과

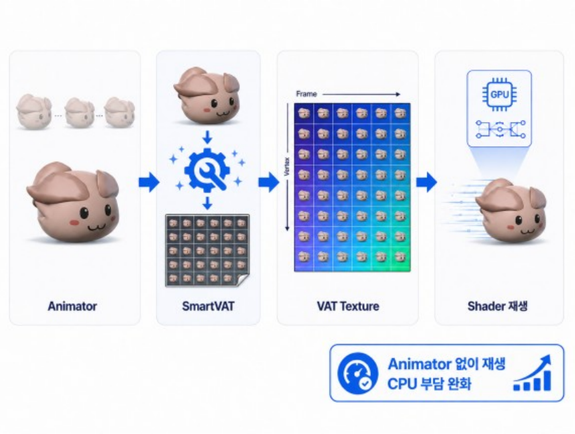
NavMesh 없이도 수백 마리 Enemy가 플레이어를 안정적으로 추적하며, **다층 지형 대응과 연산 최적화를 동시에 달성**

몬스터 최적화 - VAT + GPU Instancing

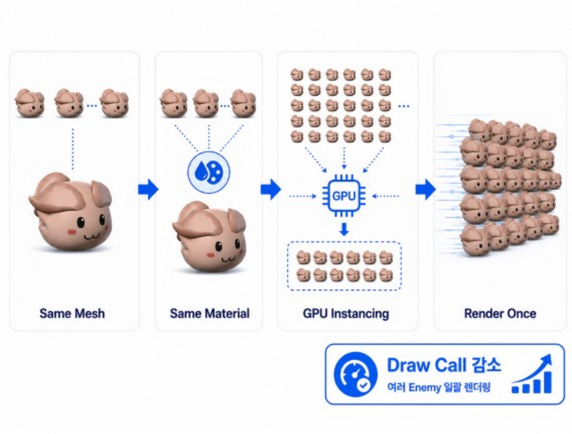
※ 문제

대량 Enemy에서 **Animator + SkinnedMeshRenderer 기반 애니메이션 처리 비용**이 누적되어 프레임 저하 발생

※ 해결



SmartVAT툴로 애니메이션을 Texture로 Bake하고, Shader에서
프레임 단위로 샘플링하여 Animator 없이 재생



Mesh·Material을 통일하고 **GPU Instancing을 적용**해
동일한 Enemy의 **렌더링 호출을 일괄 처리**

※ 결과

Animator 기반 **애니메이션 처리 비용 감소 + Draw Call 감소**로 대량 Enemy **렌더링 성능 개선**

몬스터 (보스) - 타입별 소개

미니보스(2), 스테이지보스(6), 최종보스(1) 총 **9종의 보스 구현**

- 스테이지/파이널 보스는 각 2~3종의 스킬 패턴 보유

Mini Boss

버섯이



골렘이



Stage Boss

꿀벌이



전갈이



거미왕



산삼여왕/산삼노인



용암이



유령이



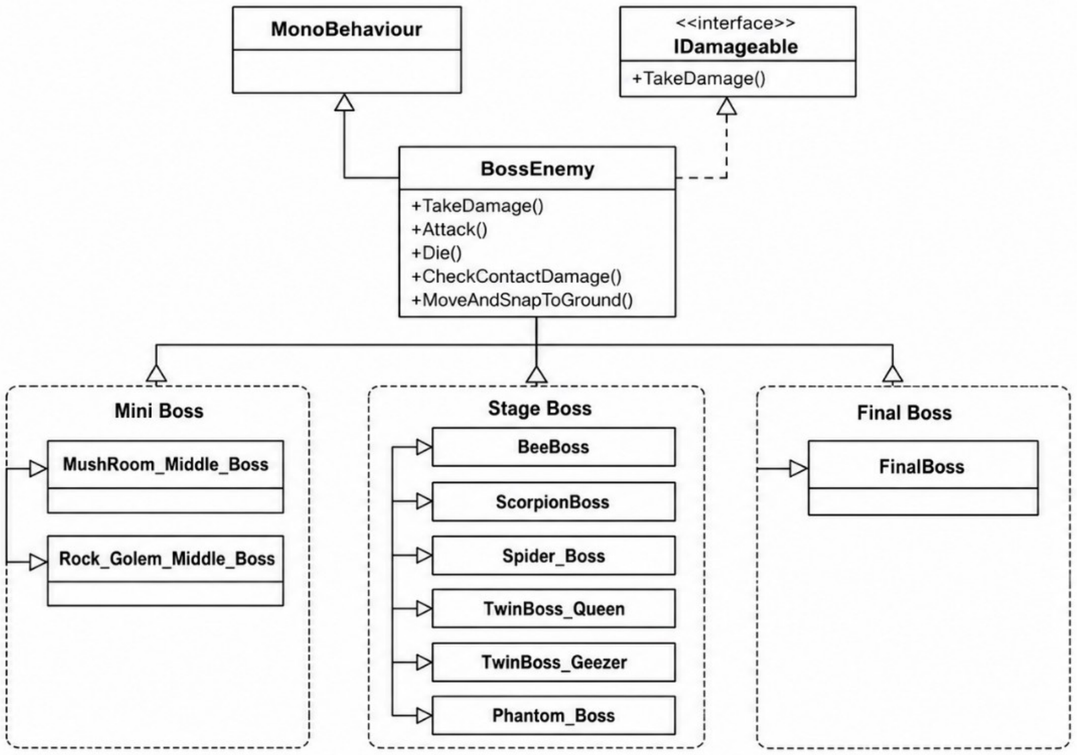
Final Boss

악몽이

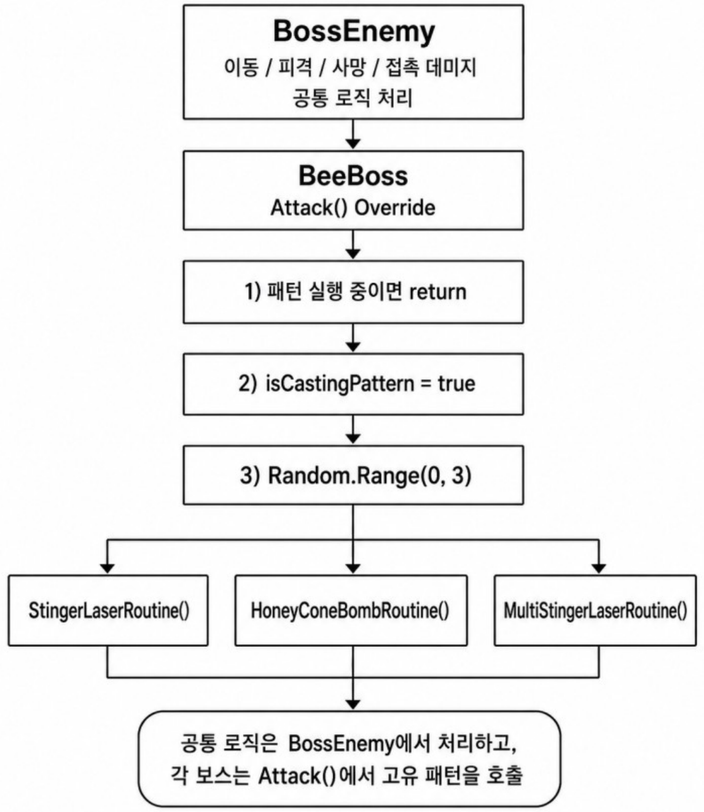


몬스터 (보스) - 상속 및 패턴 구조

BossEnemy에서 공통 로직을 관리하고, 보스별 공격 패턴은
서브클래스에서 확장하여 유지보수성과 확장성을 확보



패턴 기반 구조



몬스터 (보스) - 패턴 예고 시스템

BossTelegraph를 공통 예고 장판 모듈로 분리하여, 보스별 **패턴 확장과 전투 연출의 일관성을 확보**





핵심 효과

장판 예고, 범위 표시,
타격 판정을 공통 시스템으로 처리

→

보스별로 위치·반경·데미지·지속
시간만 설정하여
동일한 구조를 재사용

→

신규 보스 패턴 추가 시
반복 구현을 줄이고 빠르게 확장
가능

성장 시스템 기획/데이터화

민동찬

담당 역할

콘셉트 정리 + 기획

- 어린 아이들이 캐릭터 > 무기나 아이템을 아이들의 일상에서 가져오려함.
-어린이, 장난감 같은 유아틱의 컨셉을 가진 뱀서라이크
- >게임 분위기와 방향성은 밝고 아이들이 노는 놀이터 + 너무 다크하지 않게 일관성 유지 노력

데이터 구조 설계

-일관성을 위해 아이템, 무기를 장남감에서 차용 패시브 또한 뱃지로 명명
-확장 가능한 데이터 구조 설계을 위해 SO를 통한 기획을 데이터화

캐주얼 유아틱 콘셉트 기반 기획

게임 콘셉트	무기 예시	뱃지(패시브) 및 아이템 예시
- 어린아이 - 장난감 - 인형 - 간식 - 놀이 도구 밝고 친근한 분위기의 캐주얼 3D 뱀서라이크 게임	- 당근 캐논 - 선물 폭탄 - 종이비행기 - 젤리 곰 운석 칼이나 총 대신 직관적이고 귀여운 오브젝트 중심	패시브 > 뱃지: - 공격력 증가 - 쿨타임 감소 - 이동속도 증가 - 치명타율 증가 아이템-장난감, 학용품: - 소풍 배낭 - 말끔 지우개 - 코끼리 인형 - 고양이 가면

데이터의 분리

기획을 SO로 데이터화 → **런타임 구조 연결 필요** → **원본 데이터 SO와 런타임 데이터로 분리 필요**

- 플레이 중 변하는 값과 원본 데이터가 섞이면 관리 어려움
- 원본 데이터 오염 위험
- 플레이마다 독립적인 상태 관리 불가능

데이터 구조 설계

ScriptableObject 원본 데이터
- CharacterDefinitionSO
- WeaponDefinitionSO
- BadgeDefinitionSO
- ItemDefinitionSO

⇌

Runtime 상태 데이터
- PlayerRuntime
- RunSession
- PlayerInventory
- WeaponInstance
- BadgeInstance
- ItemInstance

- SO는 변경되지 않는 설계도
- 런타임은 해당 플레이 세션에서만 변하는 상태

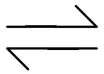
- **이유 : 상태 혼재 방지**
  - 플레이 중 상태를 SO에 저장 시 문제
  - 원본 데이터와 현재 상태 혼재
  - 다음 플레이에 영향을 줄 위험

예시 : 무기 성장 구조 - 무기 정의와 실제 보유 상태 분리

WeaponDefinitionSO

- 무기 ID
- 이름, 설명, 아이콘
- 프리팹 참조
- 레벨별 수치 데이터
- 진화 무기 ID
- 필요 뱃지 ID

변하지 않는 무기의 설계도 역할을 하며, 여러 판에서 재사용 가능



WeaponInstance

- 현재 레벨
- 현재 쿨타임
- 최대 레벨 도달 여부
- 진화 완료 여부

한판에서의 무기를 관리

>>아이템, 뱃지도 마찬가지

어려웠던점

민동찬

초기에 맡은 역할: UI 작업 담당

기본 구조 활용

- 팀장님이 'GameManager', 'UIManager', 'ServiceBootstrapper', 'ServiceLocator', 'MonoSingleton' 등 기본 구조 제공
- UI 관련 틀을 먼저 만들어주심
- 이 구조를 활용해 UI 구현하는 역할 담당

구조 이해의 필요성

- 단순히 버튼 만들고 화면 띄우는 수준 아님
- 기존 구조를 이해하고 그 위에 UI 올리는 작업 필요
- 전체 시스템과의 연결 고려

기존 구조 위에 UI 구

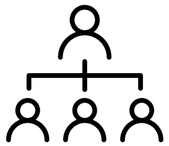
- 기존에 만들어진 구조를 기반으로 작업
- 새로운 UI 요소를 기존 시스템에 통합
- 전체 아키텍처와 조화를 이루는 UI 개발

내 역량 이상의 Task

- UI 작업이지만 코드 구조 이해 필수
- 매니저 시스템과의 상호작용 파악
- 단순 구현이 아닌 설계 이해 요구됨

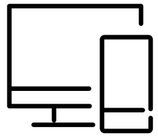
첫 번째 어려움: 기존 구조 이해

가장 먼저 어려웠던 점은 기존 프로젝트 구조를 이해하는 것이었습니다. 'UIManager' 안에는 'UniTask', 비동기 처리, 서비스 등록, UI 캐싱 같은 개념이 사용되고 있었습니다. 내 실력으로는 처음에 이 구조를 바로 이해하기 어려웠 다. AI를 활용해 이해하려고 했지만 완전히 이해했다기보다는 전체 흐름을 어렴풋하게 파악하는 데만 며칠이 걸렸습니다.



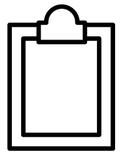
UniTask와 비동기 처리

- Unity에서 비동기 프로그래밍을 효율적으로 처리
- 'UIManager' 내부에서 사용됨
- 개념 이해가 어려웠음



서비스 등록 및 UI 캐싱

- 서비스 등록 구조 파악 필요
- UI 캐싱 시스템 이해
- 전체 구조의 복잡성



AI 활용의 한계

- AI로 이해 시도했으나 완전한 이해 어려움
- 전체 흐름 파악에 일주일 소요
- 초반 UI 작업 지체 발생

두 번째 어려움: UI 디자인

>시스템 연결의 복잡성

- 팀 프로젝트에서는 다른 시스템과 연결 필수
- 여러 매니저와의 상호작용
- 연결로 인한 난이도 증가
- 전체 구조 고려 필요

디자인 자체의 어려움

-아무것도 없는 백지 상태에서 무에서 유를 만드는 것의 어려움

-디자인 감각의 부재로 인한 힘듦.

역할 변화: 무기와 아이템 기획

데이터화 작업과 시스템 연결

데이터를 UI와 인게임에서 사용	여러 매니저와 시스템 연결	전체 매니저 구조 수정
- 무기와 아이템 데이터를 ScriptableObject와 데이터 테이블 형태로 정리 - UI 화면에서 무기 정보를 표시하기 위한 데이터 연결 작업 - 인게임 전투 시스템에서 무기 능력치를 불러오는 구조 구현 - 기존 UIManager와 새로운 데이터 시스템 간의 통신 필요	- UIManager, GameManager 등 기존 매니저들과 데이터 연동 - 서비스 등록 구조를 통한 데이터 접근 방식 적용 - 각 시스템 간 데이터 흐름 설계 및 구현 - 매니저 간 의존성 관리의 어려움 발생	- 새로운 무기 시스템 추가로 기존 구조 변경 불가피 - 각 매니저의 역할 범위가 불명확한 상태에서 작업 진행 - 전체 프로젝트 구조에 영향을 미치는 수정 발생 - 구조적 문제의 시작점이 됨

악순환 발생

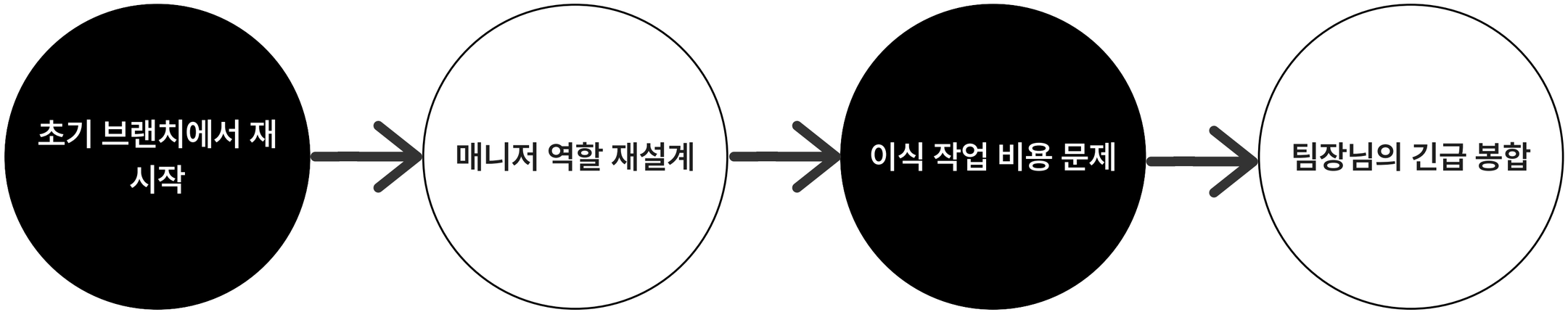
기존 구조 이해 부족 → AI를 활용해 코드 작성 → AI 코드도 완전히 이해 x → 이해에 시간 소요 ↑

선복붙 후이해 → AI 제안을 검토 없이 적용 → 기능 추가 때마다 다시 AI 의존 → AI를 활용해 이해 + 시간 소요↑↑

시간 부족 심화 → 선복붙 (후이해??)

내 품을 떠나버린 코드들

문제 해결을 위한 시도



- 문제 인식 후 초기 main 브랜치에서 새 브랜치 생성
- 복잡해진 구조를 정리하기 위한 시도
- 처음부터 다시 설계해보는 접근- 단계별 접근

- 각 매니저가 담당해야 할 책임 고민과 재정의 ..
- 데이터 흐름을 명확하게 재설계
- 클래스 간 의존성 최소화 시도

- 새로 설계한 구조를 진행된 프로젝트에 적용하기 어려움
- 이미 많은 기능이 구현된 상태
- 해결하기엔 작업량과 시간 부족

이후 느낀점과 교훈

AI 사용에 주의하자라는 단순한 결론
무조건 구조 설계때 미리 필요한 클래스들의 역할 분배와 확장성이 좋다고 생각
하지만 이번에 필요 이상으로 크게 벌려놓으니까 구조 역할 분배에 난항 > 구조 설계의 신중함

>>이번 시행착오는 실패가 아니라 팀 프로젝트에서 구조 이해,
점진적 구현과 AI 활용 방식의 중요성을 체감한 소중한 경험

시연

Q&A

감사합니다.